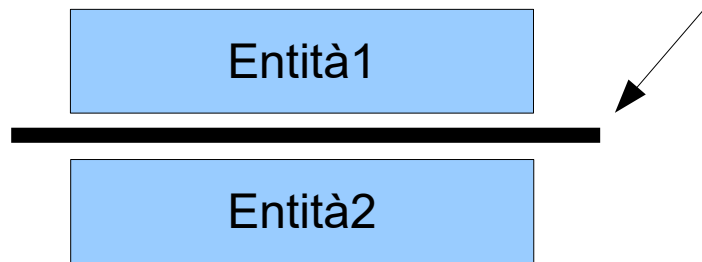


Elaborazione di file XML in Java con l'API DOM

Cos'è una API ?

API = Application Programming Interface

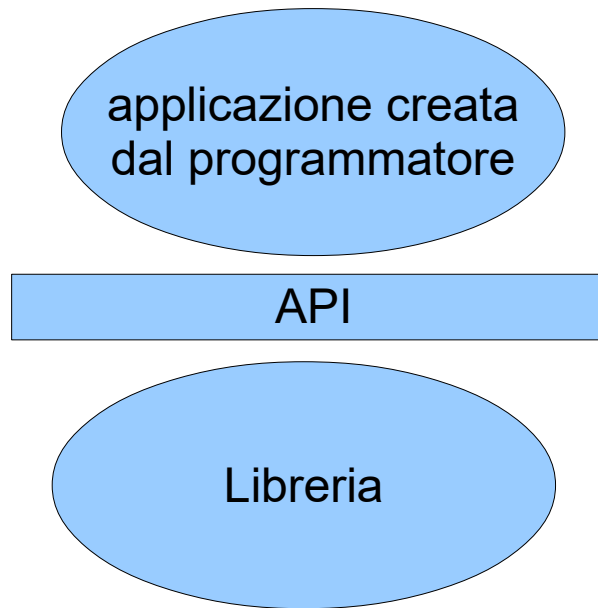
(interfaccia per la programmazione delle applicazioni)



in generale, un' interfaccia è il "mezzo" (o "superficie di separazione") attraverso cui due entità comunicano

- utente/computer (GUI)
- dispositivi hardware
- software applicativo/librerie

Cos'è una API (segue)



Una API è l'interfaccia attraverso cui i programmi applicativi possono accedere ad una libreria.

Un' API quindi è costituita dalla *descrizione* di un insieme di classi e metodi (specificando parametri, valori di ritorno, funzione svolta), ma NON specifica la loro implementazione.

Una libreria è un insieme di classi e metodi, che implementano un'API.

La stessa API può essere implementata da librerie diverse, anche in linguaggi diversi!

L'API DOM

DOM = Document Object Model

E' un' API per la manipolazione di file XML definita dal **W3C** (**World Wide Web Consortium**, ente che definisce gli standard del web come XML e HTML)

E' nata nel contesto di ECMAScript (Javascript), ma e' definita in modo **indipendente dal linguaggio di programmazione**

La sua importanza risiede nel fatto che per tutti i linguaggi di programmazione moderni esistono librerie che implementano l'API DOM

In Java L'API DOM è disponibile attraverso il package *org.w3c.dom*

L'API DOM

- Per comprendere l'API occorre ricordare la doppia rappresentazione di un file XML: testuale e ad albero
- Un albero XML è costituito da nodi di vario tipo: nodo radice, elemento radice e altri elementi, attributi, testo, commento
- Ogni nodo è un oggetto che implementa l'interfaccia **Node**
- Useremo 5 tipi di nodo, che corrispondono ad altrettante sottointerfacce definite nel package *org.w3c.dom* :

Document Element Attr Text Comment

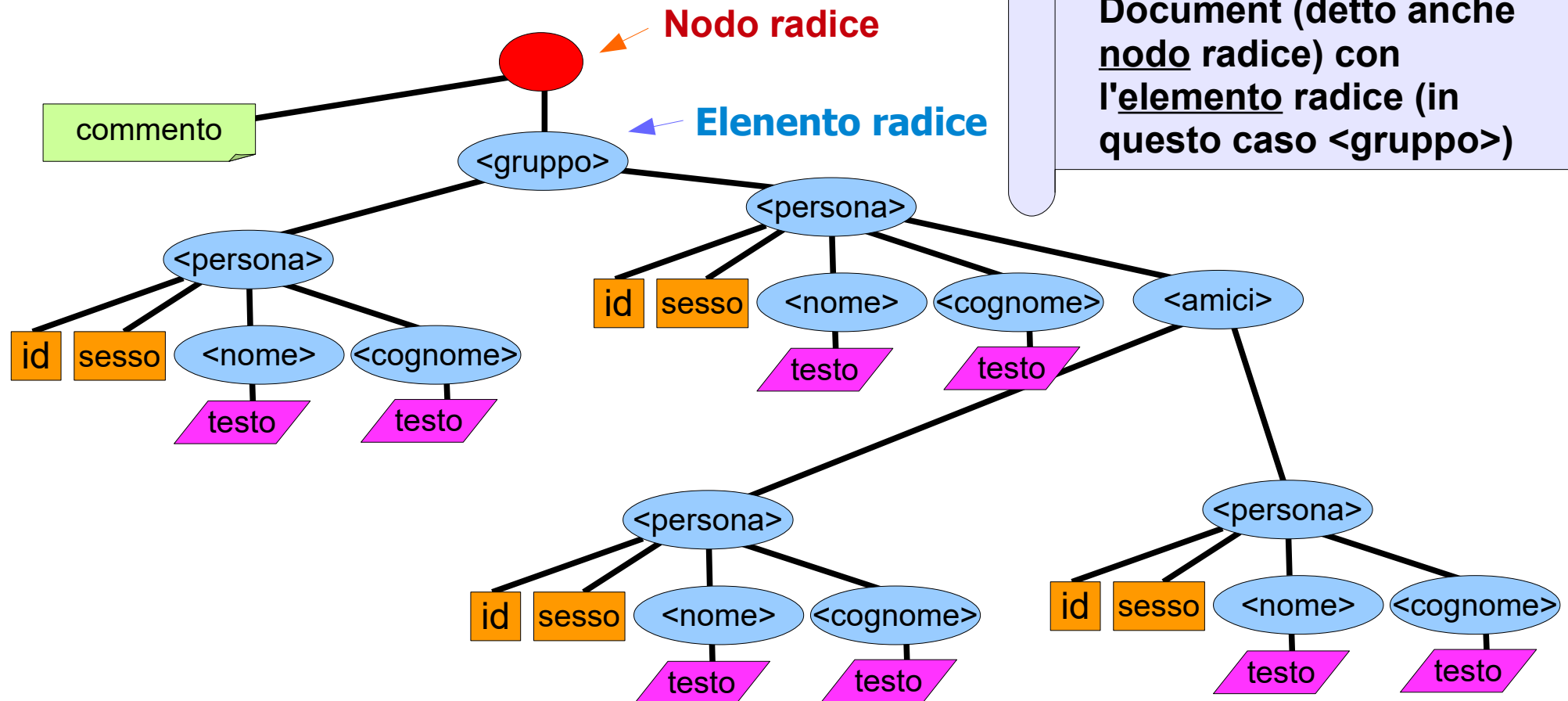
Esempio

Esempio: l'albero seguente è costituito 32 oggetti Node, di 5 diversi **tipi**:

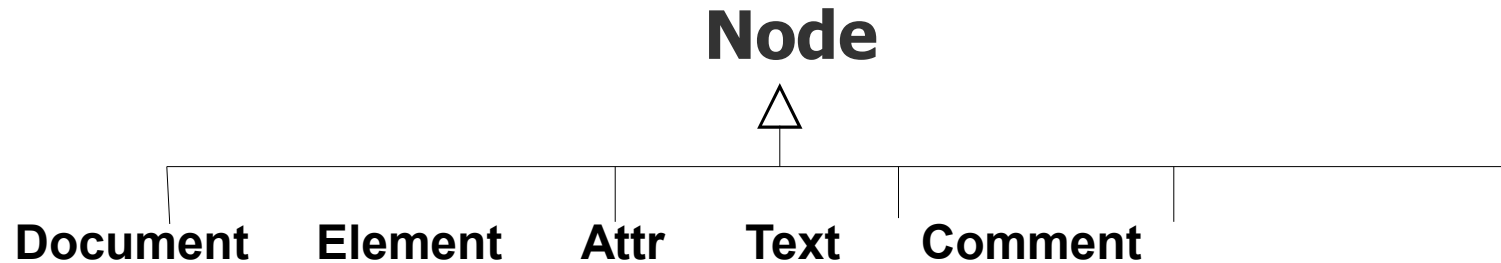
32
oggetti
di tipo
Node

- 1 nodo di tipo **Document** (ce n'è sempre e soltanto uno in ogni documento XML)
- 1 nodo di tipo **Comment**
- 14 nodi di tipo **Element**
- 8 nodi di tipo **Attr**
- 8 nodi di tipo **Text**

Attenzione: non confondere il nodo di tipo Document (detto anche nodo radice) con l'elemento radice (in questo caso <gruppo>)



Tipi di Nodi



- Le 5 interfacce Document, Element, Attr, Text e Comment sono tutte **sottointerfacce** della classe Node (cioè estendono l'interfaccia Node)
- un elemento sarà un oggetto che soddisfa sia l'interfaccia Node, sia l'interfaccia Element (più specifico)
- in altre parole: un Element è anche un Node (ma non viceversa)

DOM – Interfacce principali

Interfaccia	Descrizione
Node	E' l'interfaccia principale dell'intero package. Rappresenta un nodo generico dell'albero XML. Tutte le interfacce riportate sono due sottointerfacce (cioè estendono l'interfaccia Node).
Document	Rappresenta il nodo radice dell'intero albero XML. Può avere un solo figlio di tipo Element, ma potrebbe avere anche un figlio di tipo Comment.
Element	Rappresenta un nodo di tipo elemento (eventualmente l'elemento radice)
Attr	Rappresenta un nodo di tipo attributo
Text	Rappresenta un nodo di tipo testuale (foglia dell'albero XML).
Comment	Rappresenta un nodo di tipo commento (<!-- -->)

<https://docs.oracle.com/javase/7/docs/api/org/w3c/dom/package-summary.html>

L'interfaccia *Node*

Metodi

- String **getTextContent()**; restituisce il contenuto del nodo
- Node **appendChild(Node n)**; aggiunge un nuovo nodo al nodo n
- String **getNodeName()**; restituisce il nome del tag se il nodo è di tipo Element, oppure le stringhe costanti: "#text" se il nodo è di tipo TEXT, "#comment" se il nodo è di tipo COMMENT, "#document" se il nodo è di tipo DOCUMENT, ecc..
- Node **getParentNode()**; restituisce il nodo superiore (genitore)
- NodeList **getChildNodes()**; restituisce tutti i nodi figli del nodo
- Node **getFirstChild()**; restituisce il primo nodo figlio (quello "più a sinistra")
- Node **getLastChild()**; restituisce l'ultimo nodo figlio (quello "più a destra")
- void **normalize()**; rimuove tutti gli elementi di tipo Text vuoti
- short **getNodeType()**; restituisce un numero intero che indica il tipo di nodo, corrispondente ad una delle costanti statiche definite nell'interfaccia Node (ad esempio: Node.ELEMENT_NODE, Node.ATTRIBUTE_NODE, Node.COMMENT_NODE, Node.TEXT_NODE, ecc..)

L'interfaccia *Document*

Metodi

- Element **getDocumentElement()**; restituisce l'elemento radice del documento.
- NodeList **getElementsByTagName**(String nome-tag); restituisce la lista di tutti gli elementi del documento con un dato nome di tag.
- Element **createElement**(String nome-tag); crea un nuovo elemento con il nome dato.
- Element **getElementById**(String id); restituisce l'elemento con id passato (attenzione: non è sufficiente che l'attributo si chiami "id" affinché venga riconosciuto come identificatore)

L'interfaccia *Element*

Metodi

- String **getAttribute**(String name); restituisce il valore di un attributo in base al nome dell'attributo
- NodeList **getElementsByTagName**(String tagname); restituisce la lista degli elementi discendenti del nodo e con nome uguale a quello passato
- boolean **hasAttribute**(String name); restituisce true se l'elemento possiede un attribute con il nome passato
- void **setAttribute**(String name, String value); aggiunge un nuovo attributo

Esempio 1: lettura di file XML

Il programma seguente mostra come leggere da Java un file XML contenente un elenco di persone, e stampare tutti i cognomi presenti

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!-- un commento -->
3  <gruppo>
4    <persona id="1" sesso="m" >
5      <nome>Mario</nome>
6      <cognome>Rossi</cognome>
7    </persona>
8    <persona id="2" sesso="f" >
9      <nome>Anna</nome>
10     <cognome>Bianchi</cognome>
11     <amici>
12       <persona id="3" sesso="m" >
13         <nome>Luca</nome>
14         <cognome>Neri</cognome>
15       </persona>
16       <persona id="4" sesso="f" >
17         <nome>Elisa</nome>
18         <cognome>Verdi</cognome>
19       </persona>
20     </amici>
21   </persona>
22 </gruppo>
```

output

Rossi
Bianchi
Neri
Verdi

INPUT.xml

Soluzione Esempio 1 (passo 1)

Per prima cosa importiamo i package necessari per lavorare con i file, con l'API DOM, e con i parser predefiniti di Java:

```
import java.io.*;
import javax.xml.parsers.*;
import javax.xml.transform.*;
import javax.xml.transform.dom.*;
import javax.xml.transform.stream.*;
import org.w3c.dom.*;
```

Ricorda: un parser è un programma in grado di leggere la **rappresentazione testuale** di un file XML (es: una stringa o un file di testo) e produrre la **rappresentazione ad albero** del file XML.

Soluzione Esempio 1 (passo 2)

- Ora dobbiamo ottenere un parser DOM, che in java è un'istanza della classe **DocumentBuilder**.
- Questa classe però non ha un costruttore pubblico. Per ottenere un'istanza di DocumentBuilder dobbiamo prima ottenere un'istanza della classe **DocumentBuilderFactory** (letteralmente "fabbrica di documentBuilder") e poi da questa ottenere un parser.
- Ecco le due righe necessarie per ottenere un parser:

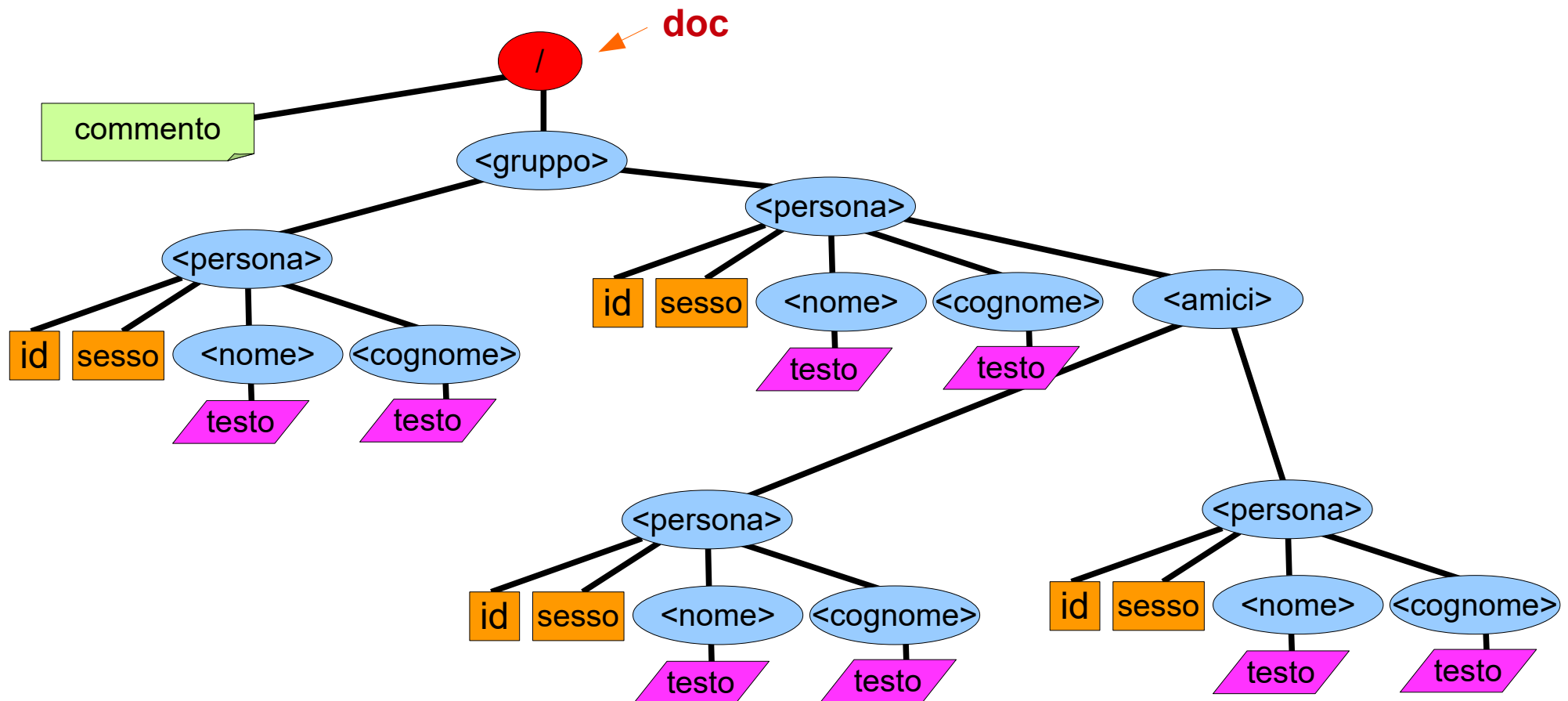
```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();  
DocumentBuilder parser = factory.newDocumentBuilder();
```

Soluzione Esempio 1 (passo 3)

- Ora che abbiamo il parser, lo usiamo per trasformare il file XML (rappresentazione testuale) in un albero, che nel nostro programma è rappresentato dalla variabile **doc**, istanza della classe Document:

```
File f = new File("input.xml");
```

```
Document doc = parser.parse(f); // doc è la radice dell'albero mostrato sotto
```



Osservazioni

La variabile **doc** "punta" alla radice dell'albero. L'albero è costituito da nodi di diverso tipo. L'albero in figura contiene 5 diversi **tipi di nodo**:

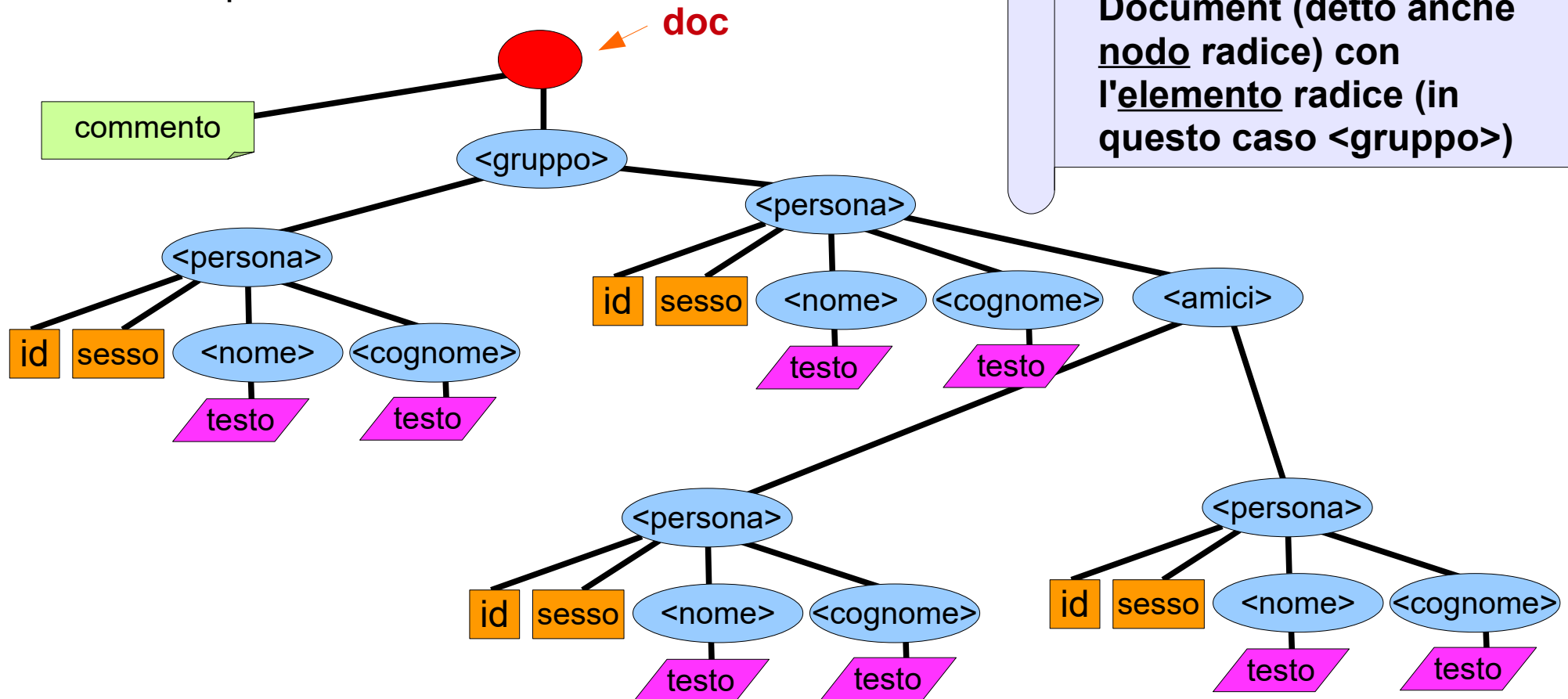
1 nodo di tipo **Document** (ce n'è sempre e soltanto uno in ogni documento XML)

1 nodo di tipo **Comment**

14 nodi di tipo **Element**

8 nodi di tipo **Attr**

8 nodi di tipo **Text**



Soluzione Esempio 1 (Passo 4)

- Ora che abbiamo il nodo radice del documento (doc) dobbiamo "scendere" sull'elemento radice. Per farlo si usa il metodo `getDocumentElement`

```
Element el = doc.getDocumentElement();
```

- Ora la variabile **el** rappresenta l'elemento `<gruppo>`
- Possiamo averne una conferma stampando il nome dell'elemento col il metodo **`getNodeName()`**

```
System.out.println( el.getNodeName() ); // stampa: gruppo
```

Soluzione Esempio 1 (Passo 5)

- Ora che abbiamo l'elemento radice (el) otteniamo la lista degli elementi "cognome" tramite il metodo **getElementsByTagName**
- Questo metodo restituisce un oggetto di tipo **NodeList** contenente la lista di tutti i discendenti del tipo richiesto

```
NodeList list = el.getElementsByTagName("cognome");
```



cerca fra tutti gli elementi **discendenti** di **el**
(quindi i suoi figli, i figli dei figli, ecc..) e restituisce
solo gli elementi con nome uguale a quello passato

Soluzione Esempio 1 (Passo 6)

- La classe `NodeList` consente di accedere ad un elenco di oggetti di tipo **Node**. Ogni nodo corrisponde in questo caso ad un elemento `<cognome>`
- Attenzione: la classe `NodeList` **non** implementa l'interfaccia `java.util.List` di Java, quindi non si possono usare gli usuali metodi di `List` per scorrerla
- Per scorrere la `NodeList` occorre usare i suoi metodi ***getLength***, che restituisce il numero di elementi, e ***item(int i)*** che restituisce l'elemento in posizione *i*

```
for( int i=0; i<list.getLength(); i++) {  
    Node n = list.item(i);  
    String contenuto = n.getTextContent();  
    System.out.print( contenuto+" ");  
}
```

Programma completo

```
package Esempio01;
import java.io.*;
import javax.xml.parsers.*;
import org.w3c.dom.*;
public class Esempio01 {
    public static void main(String[] args) throws Exception {
        DocumentBuilderFactory factory =
            DocumentBuilderFactory.newInstance();
        DocumentBuilder parser =
            factory.newDocumentBuilder();
        File f = new File("input.xml");
        Document doc = parser.parse(f);
        Element el = doc.getDocumentElement(); // elemento radice
        System.out.println( el.getNodeName() ); // stampa "gruppo"
        NodeList list = el.getElementsByTagName("cognome");
        for( int i=0; i<list.getLength(); i++) {
            Node n = list.item(i);
            String contenuto = n.getTextContent();
            System.out.print( contenuto+" ");
        }
    }
}
```

Esercizi

Utilizzando come input un file XML con la stessa struttura del file dell'esempio 1, scrivere i seguenti programmi:

Es 1. Scrivi un programma che stampa l'id della persona il cui nome è Anna

Es 2. Scrivi un programma che stampa tutti e soli i nomi delle persone di sesso maschile.

Es 3. Scrivi un programma che stampa tutti e soli i nomi delle persone che hanno almeno un amico (si suppone che l'elemento `<amici>`, se presente, contenga sempre almeno una persona)

Es 4. Scrivi un programma che legge il file `meteo.xml` e stampa la temperatura media registrata nel mese di agosto 2015. Il formato del file `meteo.xml` è riportato a pagina seguente. Si può supporre che gli elementi `<temperatura>` e `<umidita>` compaiano sempre nello stesso ordine, e che l'elemento `temperatura` sia obbligatorio.

Es 5. Analizzando il file `meteo.xml`, scrivi un programma che stampa tutti i giorni in cui non è presente la misurazione dell'umidità dell'aria

Esercizi

```
<?xml version="1.0" encoding="UTF-8" ?>
<rilevazioni>
  <rilevazione data='2015-01-01' >
    <temperatura> 2.0 </temperatura>
  </rilevazione>
  <rilevazione data='2015-08-05' >
    <temperatura> 30 </temperatura>
    <umidita> 98 </umidita>
  </rilevazione>
  <rilevazione data='2015-08-10' >
    <temperatura> 40 </temperatura>
  </rilevazione>
  <rilevazione data='2015-12-31' >
    <temperatura> 4.5 </temperatura>
    <umidita> 85 </umidita>
  </rilevazione>
</rilevazioni>
```

File meteo.xml
per gli esercizi
4 e 5